BIG DATA LAB · CASE STUDY REPORT

Merchandise Categorization and Inventory

Integrating MongoDB and Neo4j through XML

A mediator that merges catalog, stock and category data into one XML document validated by an XML Schema

Name _____

Register number _____

Course _____

Institution _____

October 2026

The case study is modelled on an online beauty retailer. It is not affiliated with Nykaa. Prices, stock and warehouses are illustrative.

Table of Contents

3 Scenario Description 3
3.1 Background 3
3.2 Problem Statement 3
3.3 Objectives 3
3.4 Scope and Sample Data 4
3.5 Approach in Brief 4
4 Requirements 5
4.1 Functional Requirements 5
4.2 Non-Functional Requirements 5
4.3 Software and Environment Requirements 6
5 System Architecture Diagram 7
5.1 Search Flow 8
5.2 Design Decisions 8
6 Database Design 9
6.1 MongoDB Database Design 9
6.2 Neo4j Graph Database Design 10
6.3 XML Schema Design 12
7 Implementation 15
7.1 Project Structure 15
7.2 Environment Setup 15
7.3 Loading the Data 15
7.4 Database Access and the Trace 16
7.5 XML Construction, Merge and Validation 17
7.6 Web API 17
7.7 Frontend 18
7.8 Testing 18
7.9 Problems Encountered and Resolutions 19
8 Results and Discussion 20
8.1 Test Searches 20
8.2 Screenshots of the System 21
8.3 Behaviour When a Database Fails 24
8.4 Schema Validation 24
8.5 Discussion 24
9 Conclusion 26

3 Scenario Description

3.1 Background

Online beauty and personal-care retailers such as Nykaa sell thousands of products across makeup, skin care, hair care and fragrance. Behind the storefront, several teams depend on the same product data. Merchandisers organise products into categories and decide which items are shown together. Warehouse teams watch how many units of each product are left in each fulfilment centre. Customer-support staff need both answers at once when a customer asks about an item.

This project is an academic case study modelled on that setting. It is not affiliated with Nykaa. The brands and product names are real, but all prices, stock quantities and warehouses are invented for illustration.

3.2 Problem Statement

The information needed to describe a single product naturally falls into three kinds of data, and each suits a different data model:

- **Catalog.** Name, brand, price, tags and attributes that differ by product type (a shade for a lipstick, an SPF for a sunscreen, a hair type for a shampoo). This data is uneven and document-shaped.
- **Inventory.** One record per product per warehouse, with a quantity and a reorder level. These are many small records that must obey strict rules.
- **Categorisation.** A category tree (Makeup > Lips > Lipstick) and links between products (bought together, similar). This data is made of relationships.

A user should not have to query three places and combine the answers by hand. The system must therefore ask each database for the data it is best at, bring the answers together using a shared key (the SKU), and present one consistent result, even when one source is missing or unavailable.

3.3 Objectives

- Store each kind of data in the database whose model fits it best: MongoDB for the catalog and stock, Neo4j for categories and relationships.
- Use XML as the common exchange format. Each database result becomes an XML document, and the documents are merged by SKU.
- Define an XML Schema (XSD) and validate every merged document against it.
- Provide one web page where a user searches by SKU or category and sees the catalog data, stock, category path and related products.
- Record a trace of every database call so the behaviour of the integration is visible.
- Keep answering, with reduced data, when one of the databases is down.

3.4 Scope and Sample Data

Table 1: Size of the sample data set

Item	Amount	Notes
Products	18	11 product types, such as lipstick, foundation, serum, sunscreen, shampoo and perfume
Inventory documents	25	Stock rows across 3 warehouses: Mumbai (WH-MUM), Delhi NCR (WH-DEL), Bengaluru (WH-BLR)
Categories	21	4 top-level categories (Makeup, Skin, Hair, Fragrance) and 17 parent-child links
Product-category links	18	Each product sits in exactly one lowest-level category
Product-to-product links	12	5 BOUGHT_TOGETHER and 7 SIMILAR_TO

The data set contains deliberate edge cases so that the system's handling of imperfect data can be demonstrated:

Table 2: Deliberate edge cases in the data

SKU	Situation	Expected behaviour
OIL-001	No stock records at all	Shown as "No stock records"; the XML is still valid
CMP-001	Quantity 0 in its only warehouse	Status OUT
FND-001, LIP-003, SUN-001, PER-002	Quantity at or below the reorder level	Status LOW

3.5 Approach in Brief

The solution follows the **mediator** pattern. A small web service sits between the browser and the two databases. For each search it asks MongoDB and Neo4j their own questions, converts each answer into an XML document, merges the two documents into one by SKU, validates the merged document against an XML Schema, and returns it to the browser together with a trace of every database call. The browser never talks to a database directly.

4 Requirements

4.1 Functional Requirements

Table 3: Functional requirements

ID	Requirement
FR1	Accept a search text that is either a SKU (for example LIP-001) or a category name (for example Makeup). Matching ignores case and surrounding spaces.
FR2	If the text is a SKU, return that product. Otherwise treat it as a category and return every product in it, including products in its subcategories.
FR3	Return catalog data from MongoDB: name, brand, price in INR, tags and the product's own attributes.
FR4	Return stock per warehouse from MongoDB with quantity, reorder level and a status (OK, LOW or OUT), plus the total quantity.
FR5	Return the category path (top-level category down to the product's category) and the related products with the type of relation, from Neo4j.
FR6	Convert the MongoDB result and the Neo4j result into separate XML documents.
FR7	Merge the two XML documents into one document, joining on the SKU.
FR8	Validate the merged document against an XML Schema and report whether it is valid, with error messages if not.
FR9	Record a trace of every database call: database, purpose, query text, parameters, number of rows, time in milliseconds and any error.
FR10	Keep answering when one database fails: return whatever the other database provided and show which call failed.
FR11	Provide one HTML page that shows the results and lets the user follow a category or a related product with one click.
FR12	Provide a health endpoint that reports whether each database can be reached.

4.2 Non-Functional Requirements

Table 4: Non-functional requirements

ID	Quality	Requirement
NFR1	Responsiveness	A search should complete in well under a second on a local machine. A database that is down must not freeze the page: connection attempts give up after about 3 seconds.
NFR2	Reliability	One failing database must not make the whole search fail (see FR10).
NFR3	Data integrity	MongoDB rejects negative or non-integer quantities and duplicate stock rows. Neo4j rejects duplicate category names and duplicate SKUs. Every merged document must pass the XSD.

ID	Quality	Requirement
NFR4	Security	Database ports are bound to the local machine only. Credentials live in a git-ignored .env file. All queries are parameterised. The page inserts server data as text, never as HTML. Search text is limited to 100 characters.
NFR5	Usability	A single page. Stock status is shown as text as well as colour. The page follows the system's light or dark theme.
NFR6	Maintainability	Small modules with heavy comments, and self-tests that can be run with one command.
NFR7	Portability	The databases start from one Docker Compose file. The Python code avoids syntax newer than Python 3.9.

4.3 Software and Environment Requirements

Table 5: Software used

Component	Version	Purpose
MongoDB	7 (image mongo:7)	Product catalog and inventory
Neo4j	5 (image neo4j:5)	Category tree and product relationships
Docker Desktop / Compose	Current	Runs both databases from one file
Python	3.9	Backend language
FastAPI and Uvicorn	0.128 and 0.39	Web service
PyMongo	4.18	MongoDB driver
neo4j (Python driver)	5.28	Neo4j driver (Bolt protocol)
lxml	6.1	Building, merging and validating XML
python-dotenv	1.2	Reads settings from .env
Web browser	Any modern browser (tested with Microsoft Edge)	Shows the single HTML page

5 System Architecture Diagram

Figure 1 shows the architecture. The browser sends one request to the mediator. The mediator is a FastAPI application made of three Python modules and one schema file. It is the only component that talks to the databases.

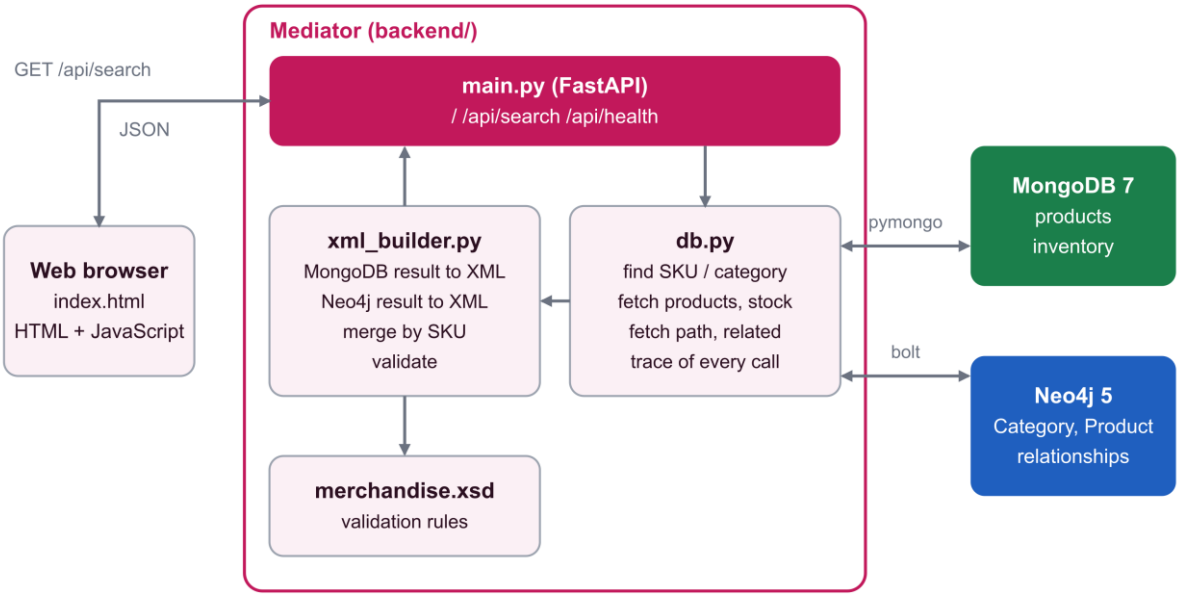


Figure 1: System architecture

Table 6: Components and responsibilities

Component	File	Responsibility
Web page	frontend/index.html	Search box, product cards, query trace and XML tabs. Calls /api/search and draws the result.
API layer	backend/main.py	Exposes /api/search and /api/health, serves the page at /. Calls db.py, then xml_builder.py.
Database access	backend/db.py	Connects to both databases, runs the queries, decides the search mode and records the trace.
XML layer	backend/xml_builder.py	Turns each database result into XML, merges the two documents by SKU and validates the result.
Schema	backend/merchandise.xsd	The rules every merged document must satisfy.
Data loader	backend/seed.py	Wipes and reloads both databases with the sample data.
MongoDB	container mongo:7	Collections products and inventory in the database nykaa.
Neo4j	container neo4j:5	Category and Product nodes with their relationships.

5.1 Search Flow

Figure 2 shows what happens for one search. The mediator first decides what the text means, then fetches the details for every SKU it found.

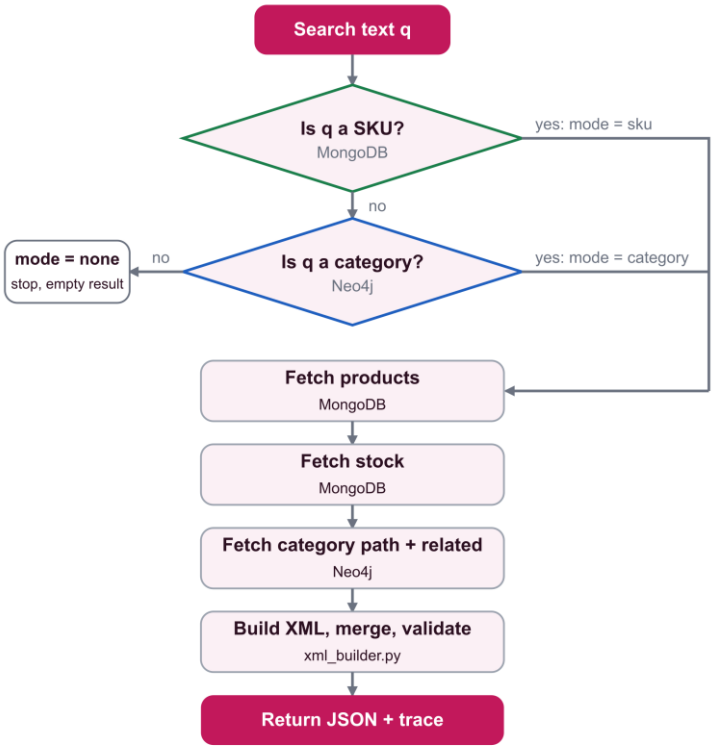


Figure 2: Flow of one search

1. **Is it a SKU?** MongoDB is asked whether a product with that SKU exists. The text is upper-cased first, so lip-001 works.
2. **Is it a category?** If not, Neo4j is asked for all products in a category with that name, including every subcategory below it. If nothing is found, the search stops with mode none.
3. **Fetch the details.** For the SKUs found, MongoDB returns the catalog data and the stock rows, and Neo4j returns the category path and the related products.
4. **Build, merge and validate.** xml_builder.py creates one XML piece per database, merges them by SKU and validates the result.
5. **Return.** The page receives the three XML documents, the validation outcome and the trace.

5.2 Design Decisions

- **Single point of access.** Only the mediator knows how to reach the databases, so the browser stays simple and no database port needs to be exposed beyond the local machine.
- **Independent XML pieces.** Each database result becomes its own XML document before merging. Neither database needs to know the other exists.
- **The SKU is the join key.** MongoDB holds the details of a SKU; Neo4j holds only the SKU and its position in the graph.
- **Failure isolation.** Every database call goes through one helper that catches errors, so a failing call yields an empty result and a trace entry instead of a crash.

6 Database Design

The design assigns each kind of data to the database that models it most naturally. MongoDB stores what a product is and how many are in stock. Neo4j stores where a product sits and how products relate to each other. The two sides share only the SKU.

6.1 MongoDB Database Design

The database is called nykaa and has two collections.

Collection products

Table 7: Fields of a product document

Field	Type	Description
sku	string	Unique product code such as LIP-001. A unique index prevents duplicates.
name	string	Product name
brand	string	Brand name
price	number	Price in INR
tags	array of strings	Labels such as bestseller or matte
attributes	object	Properties that differ by product type (see Table 8)

```
{
  "sku": "LIP-001",
  "name": "SuperStay Matte Ink Liquid Lipstick",
  "brand": "Maybelline New York",
  "price": 699,
  "tags": ["long-lasting", "transfer-proof", "bestseller"],
  "attributes": { "shade": "Pioneer", "finish": "Matte", "volume_ml": 5 }
}
```

The attributes object is the main reason the catalog lives in a document database. A relational table would need a column for every property of every product type, and most cells would be empty.

Table 8: How attributes differ by product type

Product type	Example attributes
Lipstick	shade, finish, volume_ml or weight_g
Foundation	shade, skin_type, volume_ml, spf
Sunscreen	spf, pa_rating, volume_ml
Serum	key_ingredient, skin_type, volume_ml
Shampoo	hair_type, volume_ml, key_ingredient
Perfume	notes, concentration, volume_ml

Collection inventory

Stock is stored as one document per product per warehouse.

Table 9: Fields of an inventory document

Field	Type	Rule
sku	string	Refers to a product in products
warehouse	object	Holds code (WH-MUM) and city (Mumbai); both are required
quantity	integer	Whole number, at least 0
reorder_level	integer	Whole number, at least 0

```
{ "sku": "LIP-001", "warehouse": { "code": "WH-MUM", "city": "Mumbai" },
  "quantity": 140, "reorder_level": 20 }
```

Integrity rules

MongoDB itself enforces the rules, so bad data is rejected no matter which program writes it. The collection is created with a validator:

```
{ "$jsonSchema": {
    "bsonType": "object",
    "required": ["sku", "warehouse", "quantity", "reorder_level"],
    "properties": {
      "sku":           { "bsonType": "string" },
      "warehouse":     { "bsonType": "object", "required": ["code", "city"] },
      "quantity":      { "bsonType": "int", "minimum": 0 },
      "reorder_level": { "bsonType": "int", "minimum": 0 } } } }
```

- A unique index on `products.sku` prevents two products with the same SKU.
- A unique compound index on `inventory.sku` and `inventory.warehouse.code` allows one row per product and warehouse.
- The stock **status** is not stored. It is computed when read: OUT if the quantity is 0, LOW if the quantity is at or below the reorder level, otherwise OK. This keeps the rule in one place and the stored data free of derived values.

Design decisions

- **Stock is separate from the catalog.** Stock changes far more often than product details, and one product has several warehouse rows. Separate documents can be updated and validated on their own.
- **A product may have no stock rows.** OIL-001 is in products but has no inventory documents. This is valid and is handled explicitly downstream.
- **Categories are not stored in MongoDB.** The category structure is owned by Neo4j, so there is a single source of truth for it.

6.2 Neo4j Graph Database Design

Neo4j stores the category tree and the links between products. It holds only the SKU of each product, never its details, so nothing is duplicated between the two databases.

Table 10: Graph model

Element	Meaning
(:Category {name})	A category. The name is unique.
(:Product {sku})	A product. The SKU is unique and matches the SKU in MongoDB.
(child)-[:SUBCATEGORY_OF]->(parent)	A category belongs to a parent category. Top-level categories have no parent.
(product)-[:IN_CATEGORY]->(category)	The lowest-level category the product belongs to.
(a)-[:BOUGHT_TOGETHER]->(b)	Customers often buy the two products together. Stored in one direction; queries ignore the direction.
(a)-[:SIMILAR_TO]->(b)	The two products are alternatives. Stored in one direction; queries ignore the direction.

Figure 3 shows the Makeup branch of the graph.

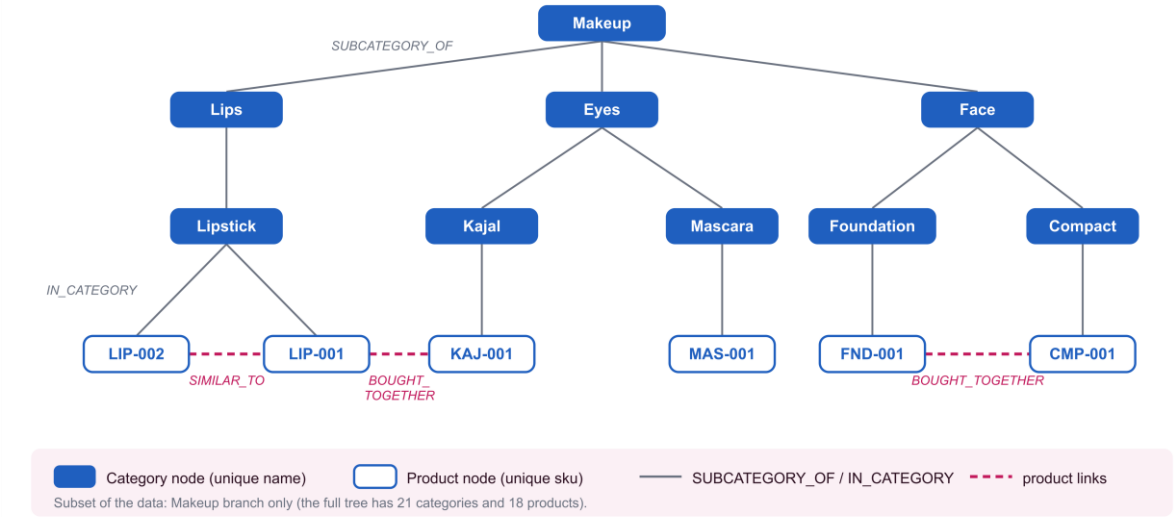


Figure 3: Graph model, Makeup branch

Table 11: Category tree (21 categories)

Top level	Second level	Lowest level (holds products)
Makeup	Lips, Face, Eyes	Lipstick; Foundation, Compact; Kajal, Mascara
Skin	Face Care	Moisturizer, Serum, Sunscreen
Hair	Hair Care	Shampoo, Hair Oil
Fragrance	Women's Fragrance	Perfume

Table 12: Product-to-product relationships in the sample data

Relationship	Pairs
BOUGHT_TOGETHER (5)	FND-001 and CMP-001; LIP-001 and KAJ-001; SER-001 and SUN-002; MOI-001 and SUN-001; SHA-001 and OIL-001
SIMILAR_TO (7)	LIP-001 and LIP-002; LIP-002 and LIP-003; FND-001 and FND-002; SER-001 and SER-002; SUN-001 and SUN-002; SHA-001 and SHA-002; PER-001 and PER-002

Constraints

- `CREATE CONSTRAINT category_name FOR (c:Category) REQUIRE c.name IS UNIQUE`
- `CREATE CONSTRAINT product_sku FOR (p:Product) REQUIRE p.sku IS UNIQUE`

Key queries

Products in a category. The pattern `*0..` means "walk down zero or more levels". Zero levels returns products placed directly in the category, and more levels include every subcategory. A search for Makeup therefore finds products in Lips, Face and Eyes without listing them.

```
MATCH (c:Category) WHERE toLower(c.name) = toLower($name)
MATCH (c)<-[:SUBCATEGORY_OF*0..]-(sub:Category)<-[:IN_CATEGORY]-(p:Product)
RETURN DISTINCT p.sku AS sku ORDER BY sku
```

Category path and related products for a list of SKUs. The query walks up the tree to the root and collects the related products. `OPTIONAL MATCH` keeps products that have no related products.

```
UNWIND $skus AS sku
MATCH (p:Product {sku: sku})-[:IN_CATEGORY]->(c:Category)
MATCH path = (c)-[:SUBCATEGORY_OF*0..]->(root:Category)
WHERE NOT (root)-[:SUBCATEGORY_OF]->()
OPTIONAL MATCH (p)-[r:BOUGHT_TOGETHER|SIMILAR_TO]-(other:Product)
WITH sku, path, other, r ORDER BY type(r), other.sku
RETURN sku,
       [n IN reverse(nodes(path)) | n.name] AS path,
       [x IN collect({sku: other.sku, type: type(r)})
          WHERE x.sku IS NOT NULL] AS related
ORDER BY sku
```

6.3 XML Schema Design

XML is the format in which the two databases meet. The mediator creates three XML documents for every search:

Table 13: The three XML documents

Document	Root element	Built from	Validated?
MongoDB piece	mongodb_result	Catalog and stock rows from MongoDB	No
Neo4j piece	neo4j_result	Category path and related products from Neo4j	No
Merged document	merchandise	Both pieces, joined on the sku attribute	Yes, against merchandise.xsd

Only the merged document is validated, because it is the one handed to the user. The two pieces are intermediate and incomplete by design. The following is the merged document for LIP-001:

```
<merchandise query="LIP-001" mode="sku" count="1">
  <product sku="LIP-001">
    <name>SuperStay Matte Ink Liquid Lipstick</name>
    <brand>Maybelline New York</brand>
    <price currency="INR">699</price>
    <tags>
      <tag>long-lasting</tag><tag>transfer-proof</tag><tag>bestseller</tag>
    </tags>
    <attributes>
      <attribute name="shade">Pioneer</attribute>
      <attribute name="finish">Matte</attribute>
      <attribute name="volume_ml">5</attribute>
    </attributes>
    <stock total="200">
      <warehouse code="WH-DEL" city="Delhi NCR" quantity="60"
                 reorder_level="20" status="OK"/>
      <warehouse code="WH-MUM" city="Mumbai" quantity="140"
                 reorder_level="20" status="OK"/>
    </stock>
    <category_path>
      <category>Makeup</category><category>Lips</category>
      <category>Lipstick</category>
    </category_path>
    <related>
      <item sku="KAJ-001" type="BOUGHT_TOGETHER"/>
      <item sku="LIP-002" type="SIMILAR_TO"/>
    </related>
  </product>
</merchandise>
```

The name, brand, price, tags, attributes and stock come from MongoDB. The category path and related items come from Neo4j. The schema in merchandise.xsd defines the structure and the value rules.

Table 14: Types and rules in merchandise.xsd

Item	Rule
merchandise (root)	Attributes query (text), mode (sku, category or none) and count (whole number); contains zero or more product elements
SkuType	Text matching the pattern `[A-Z]{3}-[0-9]{3}`, for example LIP-001
StatusType	Exactly one of OK, LOW, OUT
RelationType	Exactly one of BOUGHT_TOGETHER, SIMILAR_TO
PriceType	A decimal number that is not negative, with a required currency attribute
WarehouseType	Required attributes code, city, quantity and reorder_level (whole numbers, not negative) and status (StatusType)
StockType	Zero or more warehouse elements and a required total (whole number, not negative)
AttributeType	Text content with a required name attribute. The name is data, so one schema covers every product type.

Item	Rule
ProductType	An ordered sequence: name, brand, price, tags, attributes, stock, category_path, related. Every part is optional. Only the sku attribute is required.
Unique constraint	No two product elements may have the same sku (`xs:unique`)

The central part of the schema is the product type:

```
<xs:complexType name="ProductType">
  <xs:sequence>
    <xs:element name="name"  type="xs:string" minOccurs="0"/>
    <xs:element name="brand" type="xs:string" minOccurs="0"/>
    <xs:element name="price" type="PriceType" minOccurs="0"/>
    <xs:element name="tags" minOccurs="0"> ... </xs:element>
    <xs:element name="attributes" minOccurs="0"> ... </xs:element>
    <xs:element name="stock" type="StockType" minOccurs="0"/>
    <xs:element name="category_path" minOccurs="0"> ... </xs:element>
    <xs:element name="related" minOccurs="0"> ... </xs:element>
  </xs:sequence>
  <xs:attribute name="sku" type="SkuType" use="required"/>
</xs:complexType>

<xs:unique name="uniqueSku">
  <xs:selector xpath="product"/>
  <xs:field xpath="@sku"/>
</xs:unique>
```

Design decisions

- **Optional parts.** Because a product is assembled from two databases, nearly every part is optional. A document with MongoDB data only is as valid as a complete one. The SKU is the only required part, since it is the join key.
- **Attribute names as data.** Writing `<attribute name="spf">` instead of `<spf>` lets one fixed schema describe products whose attributes differ.
- **Restricted types.** Patterns and enumerations catch malformed SKUs, unknown stock statuses and unknown relation types.

Table 15: What the schema accepts and rejects

Document	Result
Complete product (MongoDB and Neo4j parts)	Valid
Product with no stock rows (OIL-001)	Valid: stock has total 0 and no warehouse elements
Product without Neo4j parts (Neo4j down)	Valid: category_path and related are optional
No matching product (mode none, count 0)	Valid: an empty merchandise element
A warehouse with status BROKEN	Rejected: not OK, LOW or OUT
A product with sku lip1	Rejected: does not match the SKU pattern
Two products with sku LIP-001	Rejected: violates the unique constraint

7 Implementation

7.1 Project Structure

Table 16: Files of the project

Path	Description
docker-compose.yml	Starts MongoDB 7 and Neo4j 5. Ports are published on 127.0.0.1 only.
.env.example, .env	Connection settings and the Neo4j password. .env is ignored by git.
backend/requirements.txt	Python dependencies
backend/seed.py	Loads the sample data into both databases and checks the result
backend/db.py	Connections, queries, search logic and the trace
backend/xml_builder.py	XML creation, merge and validation
backend/merchandise.xsd	XML Schema for the merged document
backend/main.py	FastAPI application
frontend/index.html	The single HTML page

7.2 Environment Setup

1. Copy `.env.example` to `.env` and choose a Neo4j password of at least 8 characters.
2. Start the databases with `docker compose up -d`. The compose file declares no volumes, so `docker compose down` removes the data. Re-run the seed script afterwards.
3. Create a Python 3.9 virtual environment and run `pip install -r requirements.txt`.
4. Load the data with `python seed.py`.
5. Start the service with `uvicorn main:app` and open http://127.0.0.1:8000/.

7.3 Loading the Data

`seed.py` is safe to run repeatedly. Each run deletes both collections and every Neo4j node, then loads everything again. For MongoDB it creates the unique index on `products.sku`, creates the `inventory` collection with its validator and its compound unique index, and inserts the documents. For Neo4j it creates the two uniqueness constraints, loads the categories and the tree with `MERGE` inside an `UNWIND`, links each product to its category, and creates the product relationships.

The script ends with a self-check that counts the loaded data and proves the rules work: it tries to insert a negative quantity into MongoDB and a duplicate category into Neo4j, and both must be rejected.

7.4 Database Access and the Trace

`db.py` creates one client per database. Both are configured to give up quickly so that an unavailable database cannot freeze the page: MongoDB uses `serverSelectionTimeoutMS=3000`, and the Neo4j driver uses `connection_timeout=3` and `max_transaction_retry_time=0`.

Every database call goes through one helper, `traced()`. It starts a stopwatch, runs the call, and appends an entry to the trace with the step number, database, purpose, query text, parameters, row count and time. If the call raises an error, the helper stores the first line of the message and returns an empty list.

```
def traced(trace, database, purpose, query, params, run):
    entry = {"step": len(trace) + 1, "database": database, "purpose": purpose,
             "query": query, "params": params}
    start = time.perf_counter()                # start the stopwatch
    try:
        result = run()
        entry["rows"] = len(result)
    except Exception as error:
        result = []
        entry["rows"] = 0
        entry["error"] = str(error).splitlines()[0]   # first line is enough
    entry["ms"] = round((time.perf_counter() - start) * 1000, 2)
    trace.append(entry)
    return result
```

Table 17: Query functions in db.py

Function	Database	Purpose
find_sku	MongoDB	Is the text a SKU? The text is upper-cased first.
skus_in_category	Neo4j	SKUs in a category, including its subcategories
get_products	MongoDB	Catalog data for a list of SKUs (`$in`)
get_stock	MongoDB	Stock rows for a list of SKUs, with the computed status
get_graph_info	Neo4j	Category path and related products for a list of SKUs

`search(text)` ties these together. It first asks MongoDB whether the text is a SKU. If not, it asks Neo4j whether the text is a category. If neither finds anything it stops, so a search for xyz costs only two calls and a blank search costs none. Otherwise it fetches products, stock and graph data for the SKUs it found.

```
skus = find_sku(text, trace) if text else []
if skus:
    mode = "sku"
else:
    skus = skus_in_category(text, trace) if text else []
    mode = "category" if skus else "none"

products = get_products(skus, trace) if skus else []
stock    = get_stock(skus, trace)    if skus else []
graph    = get_graph_info(skus, trace) if skus else []
```

7.5 XML Construction, Merge and Validation

`xml_builder.py` does not touch the databases; it only reads the dictionary returned by `search()`. It has three building steps and one checking step.

- **mongo_to_xml** creates a `mongodb_result` element with one `product` per catalog document. Attribute names become `name` attributes, and every product gets a `stock` element even when it has no stock rows.
- **neo4j_to_xml** creates a `neo4j_result` element with one `product` per graph row, containing the category path and the related items.
- **merge_xml** joins the two documents by SKU.
- **validate** checks the merged document against the schema, which is loaded once when the module is imported.

The merge copies the children of every product element into one product per SKU, MongoDB first, then Neo4j. A SKU found in only one piece is kept with the parts it has. A deep copy is used because an XML element can live in only one place in a tree.

```
merged = {}                                  # sku -> the combined <product>
for piece in (mongo_xml, neo4j_xml):         # MongoDB first, then Neo4j
    for product in piece:
        sku = product.get("sku")
        if sku not in merged:
            merged[sku] = etree.Element("product", sku=sku)
        for child in product:
            merged[sku].append(copy.deepcopy(child))

root = etree.Element("merchandise", query=query, mode=mode, count=str(len(merged)))
for sku in sorted(merged):                   # same order every time
    root.append(merged[sku])

schema = etree.XMLSchema(etree.parse(str(SCHEMA_PATH)))

def validate(document):
    is_valid = schema.validate(document)
    errors = [f"line {e.line}: {e.message}" for e in schema.error_log]
    return is_valid, errors
```

Two small details matter for correctness. Python's `True` prints as `True`, but XML expects `true`, so a helper converts booleans before they are written. The search text is placed in an XML attribute, so non-printable characters are removed first; otherwise a control character typed by a user could make the document impossible to build.

7.6 Web API

Table 18: Endpoints of main.py

Endpoint	Purpose
GET /	Serves frontend/index.html
GET /api/search?q=...	Runs one search and returns JSON. The text q is limited to 100 characters; longer input is rejected with status 422.
GET /api/health	Returns "up" or the first line of the error for each database

The search response contains the fields `query`, `mode`, `skus`, `valid`, `errors`, `mongodb_xml`, `neo4j_xml`, `merged_xml` and `trace`. The endpoint is a plain `def`, not `async def`, because the database drivers block while they wait. FastAPI runs plain functions in a worker thread, so one slow search does not stop other requests.

7.7 Frontend

The page is one HTML file with embedded CSS and JavaScript, and no framework or build step. It works as follows:

- It calls `/api/search` and parses the returned merged XML with `DOMParser`. The product cards are drawn from that XML, so the page shows exactly what the mediator built.
- Each card shows catalog data and stock (MongoDB) and the category path and related products (Neo4j). Missing parts are labelled "Not available" or "No stock records".
- Category names and related SKUs are buttons that start a new search.
- The trace is shown as a table. If any call failed, a banner names the failing database and says that only the other database's data is shown.
- Three tabs show the merged XML and the two XML pieces.
- Status pills show whether each database is up and whether the XML passed validation.
- All server data is inserted with `textContent`, never `innerHTML`, so data cannot inject HTML or script into the page.
- Opening `/#q=LIP-001` runs that search at once.

7.8 Testing

Table 19: Automated self-checks

Command	What it checks
python seed.py	Counts of loaded documents, nodes and relationships; the MongoDB validator rejects a negative quantity; the Neo4j constraint rejects a duplicate category
python db.py --check	Search by SKU and by lower-case SKU; category search includes subcategories (Makeup gives 8 products); OIL-001 has no stock; FND-001 is LOW; CMP-001 is OUT; unknown text gives mode none after two calls; blank text makes no database call
python xml_builder.py --check	Valid merged XML for a SKU, a category, a product without stock, an unknown search, and a search where Neo4j returned nothing; boolean attributes are written as true; the schema rejects a bad status, a bad SKU and a duplicate SKU

In addition, the API and the page were exercised by hand with the searches in Section 8, including a run with the Neo4j container stopped.

7.9 Problems Encountered and Resolutions

Table 20: Problems and how they were resolved

Problem	Cause	Resolution
A search took about 30 seconds when Neo4j was down	The Neo4j driver retries a failed query for 30 seconds by default	Set `connection_timeout=3` and `max_transaction_retry_time=0`
The wait was still about 9 seconds on Windows	The driver still tried twice, and each attempt tried IPv6 (::1) before IPv4	Use 127.0.0.1 instead of localhost in NEO4J_URI. The wait fell to about 5 seconds, which was accepted.
Boolean attributes appeared as True	Python converts True to the text True, but XML expects true	A helper writes booleans as true or false
Odd characters in the search text could break XML creation	The text is stored in an XML attribute	Non-printable characters are removed before the attribute is set
The validity badge could be missing when a search ran on page load	The badge was created only after the health request finished	The badge element is now part of the page HTML

8 Results and Discussion

8.1 Test Searches

Table 21 lists searches run against the seeded databases through the API. Each search was run once to warm up the connections and then measured. The time is the sum of the times of all database calls in that search. It varies from run to run on a local machine.

Table 21: Results of test searches

Search	Mode	Products	Stock rows	DB calls	XML valid	DB time (ms)
LIP-001	sku	1	2	4	Yes	162
lip-001	sku	1	2	4	Yes	90
Makeup	category	8	12	5	Yes	69
Lipstick	category	3	5	5	Yes	86
Skin	category	5	8	5	Yes	79
Hair	category	3	3	5	Yes	305
OIL-001	sku	1	0	4	Yes	58
CMP-001	sku	1	1	4	Yes	28
FND-001	sku	1	2	4	Yes	39
SUN-001	sku	1	1	4	Yes	37
xyz	none	0	0	2	Yes	25
(blank)	none	0	0	0	Yes	0

Every search produced a valid merged document, including the ones with missing data. A lower-case SKU gave the same result as an upper-case one. A category search returned products from all subcategories: Makeup returned the 8 products in Lips, Face and Eyes. A SKU search needs 4 database calls and a category search needs 5. A search that matches nothing stops after 2 calls, and a blank search makes none.

8.2 Screenshots of the System

Figure 4 shows a SKU search. The card combines MongoDB data (name, brand, price, attributes, tags and stock in two warehouses) with Neo4j data (the path Makeup, Lips, Lipstick, and two related products). The trace beneath shows three MongoDB calls followed by one Neo4j call. The status pills show both databases up and the XML valid.

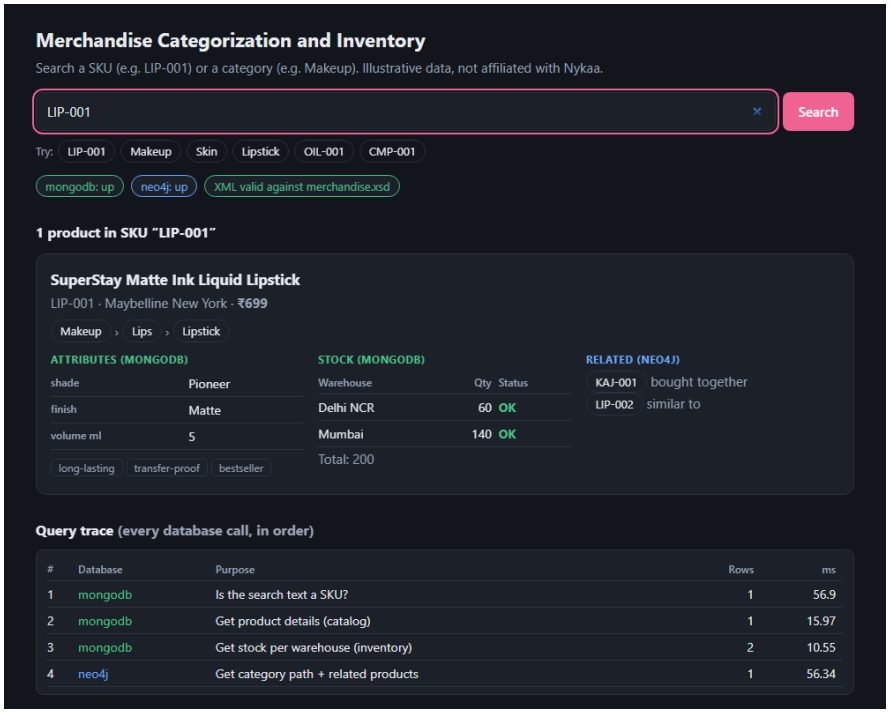


Figure 4: Search for LIP-001

Figure 5 shows a category search. Makeup returns 8 products; the first two are shown. The compact CMP-001 is OUT of stock, and the foundation FND-001 is LOW in both warehouses. Related products such as FND-001 and CMP-001 appear as links on each other's cards.

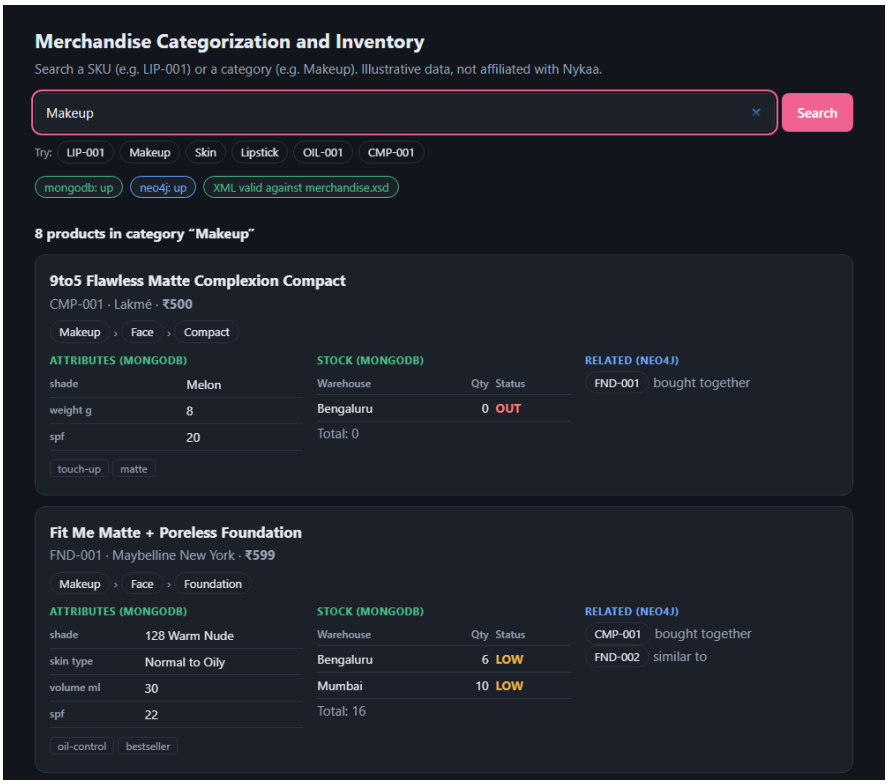


Figure 5: Search for the category Makeup (first two of eight products)

Figures 6 and 7 show the edge cases. OIL-001 has no inventory documents, so the stock area says "No stock records" while the rest of the card is complete. CMP-001 has a stock row with quantity 0, shown as OUT.

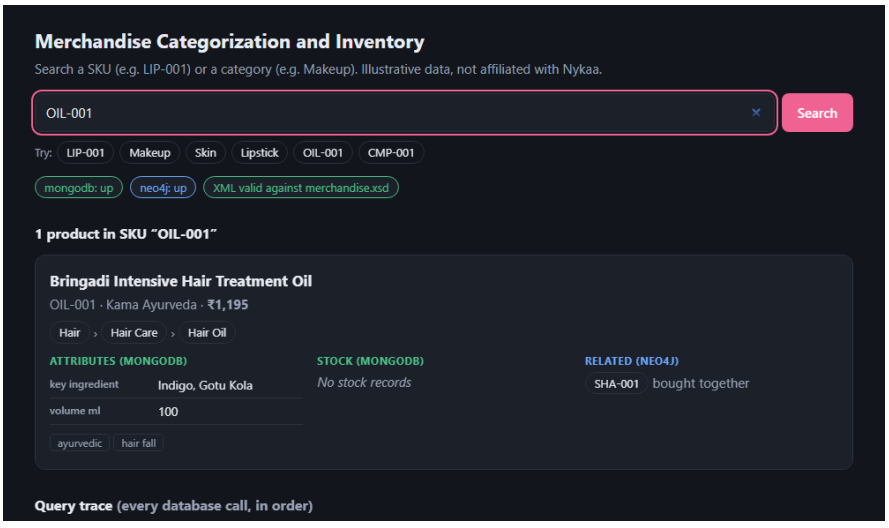


Figure 6: Search for OIL-001, a product with no stock records

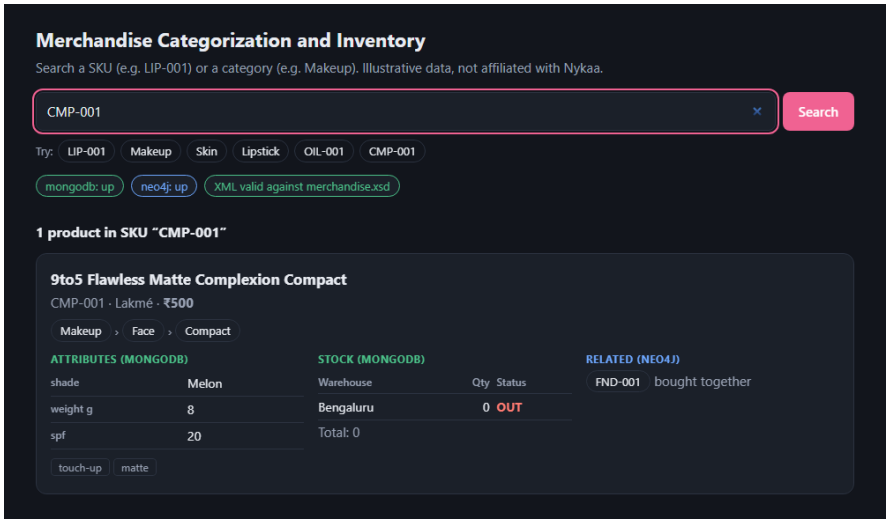


Figure 7: Search for CMP-001, out of stock

Figure 8 shows a search that matches nothing. The page says that no SKU or category matches. The trace shows exactly two calls, MongoDB and then Neo4j, and the merged XML is an empty but valid document.

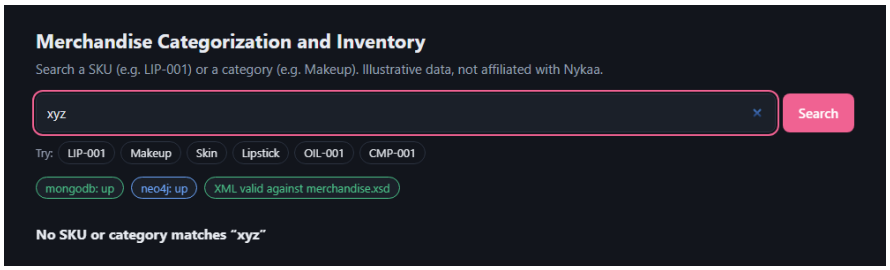


Figure 8: Search for xyz, which matches nothing

8.3 Behaviour When a Database Fails

To test failure handling, the Neo4j container was stopped and LIP-001 was searched again, as shown in Figure 9. The search still returned the full MongoDB data: name, brand, price, attributes, tags and stock. The Neo4j call (step 4) failed after about 5 seconds, and the trace recorded the error. The category area says "Category unavailable", the related area says "Not available", a banner names the failing database, and the "neo4j" status pill turns to down. The merged XML was still valid, because every Neo4j-derived part is optional in the schema.

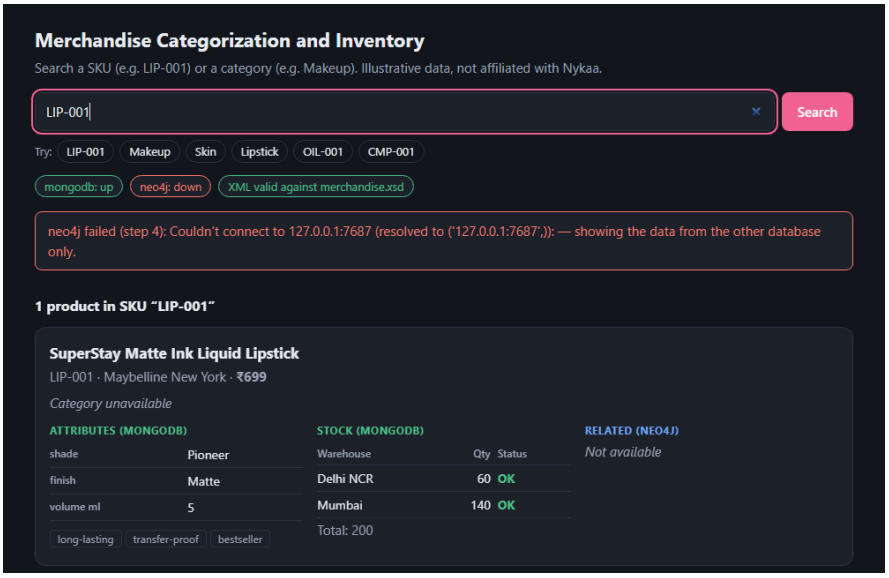


Figure 9: Search for LIP-001 with Neo4j stopped

8.4 Schema Validation

All twelve test searches in Table 21 produced a document that passed `merchandise.xsd`. The schema also rejected the three deliberately broken documents in the self-test: an unknown stock status, a malformed SKU and a duplicate SKU. Validation is therefore a real check and not a formality.

8.5 Discussion

What worked well

- **Each database does the job it suits.** Flexible attributes were easy to store in MongoDB. The Makeup search shows the benefit of the graph: one query returns products from every subcategory.
- **XML made the integration explicit.** Each database result became a separate, readable document, and the merge was a short, predictable step on the SKU.
- **The schema turned partial answers into safe answers.** Making the parts optional allowed a degraded but valid result when Neo4j was down.
- **The trace made the system explainable.** The page shows every call, the order of the calls and their cost.

Limitations

- **Exact matching only.** A search matches a whole SKU or a whole category name. There is no search by product name, brand or partial text.
- **Small data set.** The design is shown on 18 products. Result lists are not paged, and a very large category would return a very large document.
- **Sequential calls.** The database calls for one search run one after another. Calls that do not depend on each other could run in parallel.
- **No caching.** Every search goes to both databases.
- **Slow failure.** With Neo4j down, a search waits about 5 seconds on the test machine before it reports the failure.
- **No authentication.** The service is meant for a local lab. The trace shows query text, which would need protection in a public deployment.
- **Data is loaded by script.** There is no screen for changing stock or categories.

Cost of using XML

Converting to XML adds a step and makes the response larger than the same data in JSON. In return, the system gains a formal contract (the schema) and a validation result for every response. For an integration of independent sources, that contract is the main benefit of the approach.

9 Conclusion

The project built a working mediator that answers one question, "what is this product, where does it sit and is it in stock?", from two databases with different data models. MongoDB holds the flexible catalog and the stock rules; Neo4j holds the category tree and product relationships. The mediator converts each answer to XML, merges the pieces by SKU, validates the merged document against an XML Schema, and returns it with a trace of every database call to a single web page.

Each objective in Section 3.3 was met:

- Each kind of data lives in the database that fits it, with integrity rules enforced by the databases themselves.
- XML is the common format, and the merge joins on the SKU.
- `merchandise.xsd` validates every merged document, and the self-tests show it rejects bad data.
- One page supports SKU and category searches, with links between related items.
- The trace records every call, its cost and any error.
- With one database down, the system still returns the other database's data and says what failed.

The work showed that the real difficulty of combining databases lies less in reading from each one than in agreeing on a shared key, a shared format and a clear rule for what a valid, possibly incomplete, answer looks like. The schema supplied that rule.

Future work

- Search by product name, brand and partial text, and rank the results.
- Run independent database calls in parallel and cache frequent searches.
- Fail faster when a database is known to be down, for example with a quick connectivity check before the query.
- Add authentication and hide the query text from untrusted users.
- Add screens or endpoints for updating stock and editing categories.
- Store the data in persistent volumes and add automated tests that run in a continuous-integration pipeline.
- Offer the merged document as a downloadable XML file, and add XSLT transformations to other formats.